

Chapter 2

Data Organisation in Memory

Data Alignment

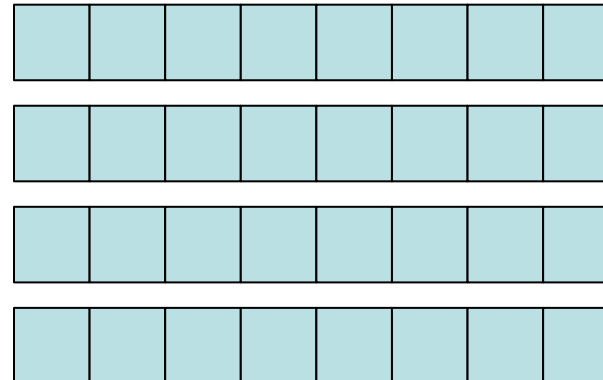
Data Organisation in Memory

Learning Objectives (2.6)

1. Describe the concept of data alignment.
2. Describe data alignment considerations for efficient access and storage of structure variables in memory.

Alignment

- Main goal: Read data in the least number of operations
 - Example: Store then read data 0xA1
 - Case 1: not aligned in 1 byte: will use two bytes
 - When reading, need to read first part then second part
 - Case 2: all in the same byte
 - Can read in a single transaction



Data Alignment

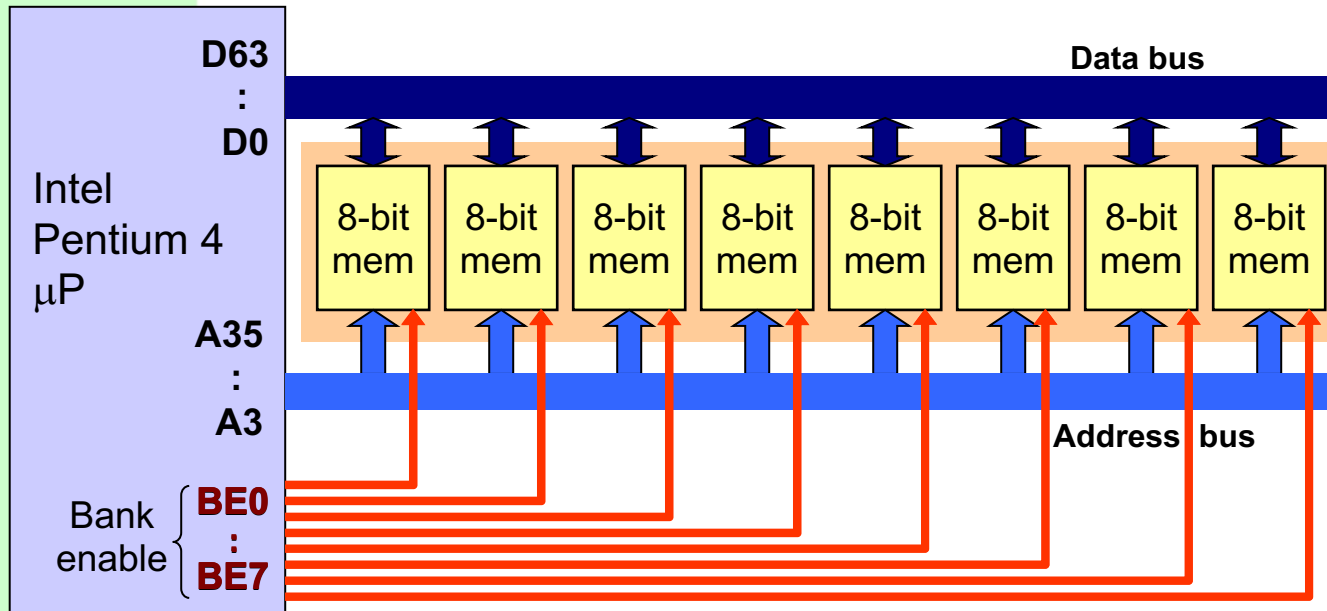
- Most computer systems have restrictions on the allowable address for accessing various data types.
- Multi-byte data (e.g. **int**, **double**) must be aligned to addresses that are multiple of values such as 2, 4, or 8.
- Programs written with Microsoft (Visual C++) or GNU (gcc) and compiled for a 64-bit Intel processor will use the following data alignment enforcement:

Data Type	Size (Byte)	Example of allowable start addresses due to alignment
char	1	0x..0000, 0x..0001, 0x..0002
short	2	0x..0000, 0x..0002, 0x..0004
int	4	0x..0000, 0x..0004, 0x..0008
float	4	0x..0000, 0x..0004, 0x..0008
double	8	0x..0000, 0x..0008, 0x..0010
pointer	8	0x..0000, 0x..0008, 0x..0010

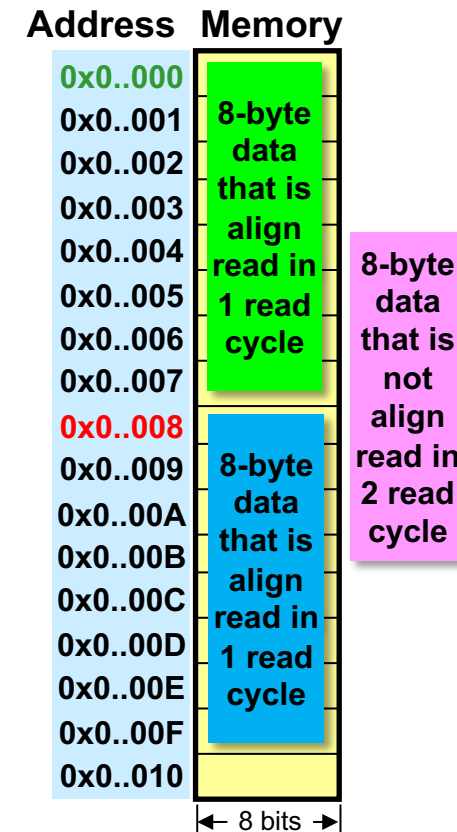
Memory Interfacing & Data Alignment

(Case Study: Intel Pentium 4)

- Main memory consist of multiple 8-bit memory modules required to make up 64-bit data word size of processor.
- Data width is selected by additional control lines (BE0-BE7).



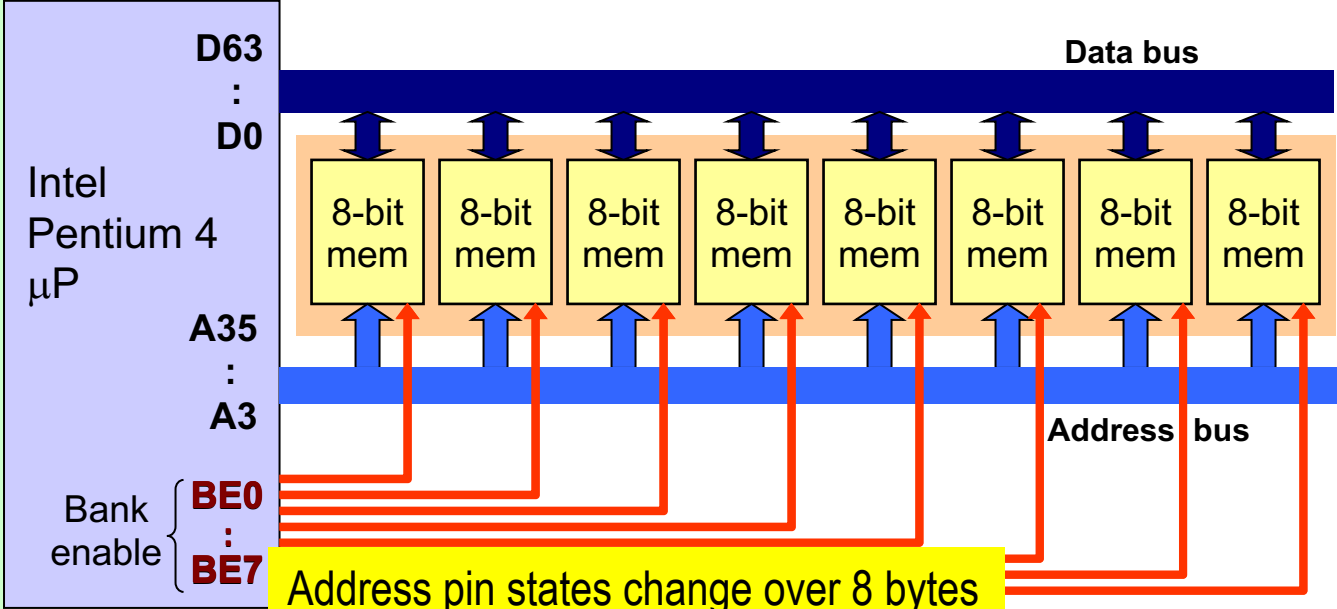
- Only one address pattern can exist on address bus during data transfer. 8-byte data needs **two memory cycles** if not aligned to 8-byte address boundary.



Learn More: Google “memory and data alignment”

Address Mapping of 64-bit data

(Case Study: Intel Pentium 4)

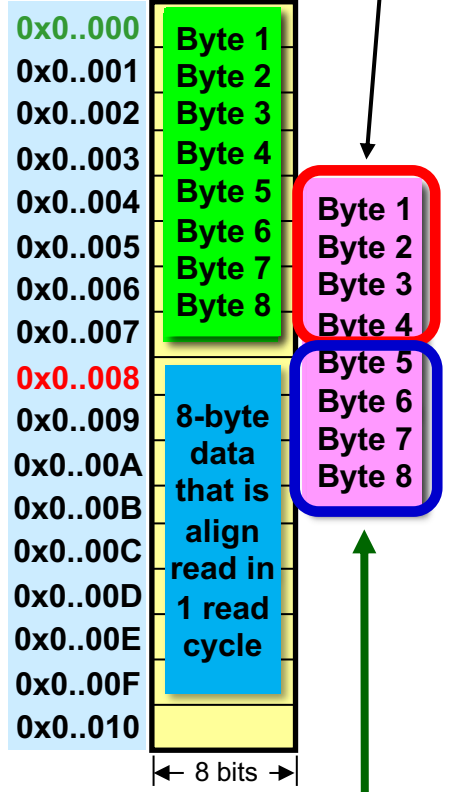


Address pin states change over 8 bytes

Address Pins

	A ₃₅	A ₃₄		A ₀₆	A ₀₅	A ₀₄	A ₀₃	A ₀₂	A ₀₁	A ₀₀
Byte 1	0	0		0	0	0	0	1	0	0
Byte 2	0	0		0	0	0	0	1	0	1
Byte 3	0	0		0	0	0	0	1	1	0
Byte 4	0	0		0	0	0	0	1	1	1
Byte 5	0	0		0	0	0	1	0	0	0
Byte 6	0	0		0	0	0	1	0	0	1
Byte 7	0	0		0	0	0	1	0	1	0
Byte 8	0	0		0	0	0	1	0	1	1

Address Memory



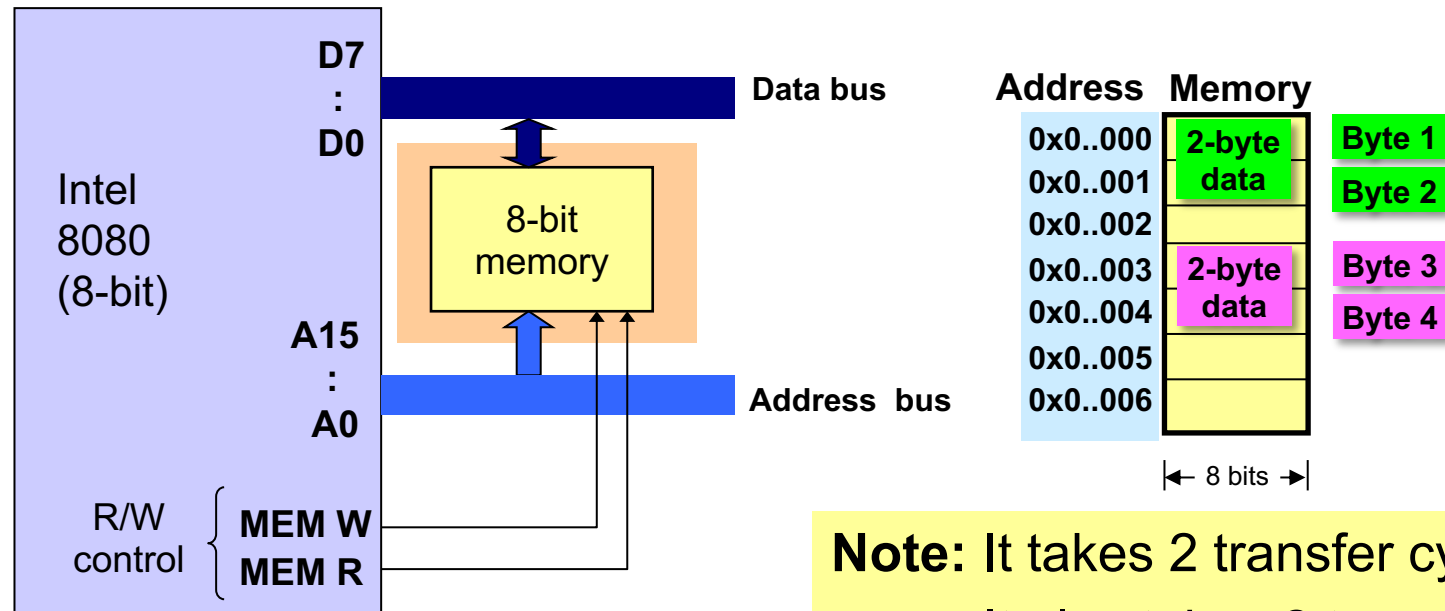
Note:
Address pin **A₀₃** cannot be in states 0 and 1 simultaneously to transfer a mis-aligned word in a single memory cycle.

Note: Lines BE0-BE7 implements address bits **A₀₂** to **A₀₀** and determine which byte(s) in the 8 possible addresses are accessed during memory transfer.

Data Alignment and Data Bus Width

(Case Study: An 8-bit Processor - Intel 8080)

- The extent of data alignment is dependent on the width of the processor's data bus.
- A processor with an **8-bit** data bus does not have any data alignment issues. Multi-byte data can be fetched from **any address**.
- A processor with a **16-bit** (i.e. 2 bytes) data bus only needs to align 2, 4 or 8 byte-sized data to **even addresses** (e.g. 0x0000, 0x0002, 0x0004, etc).



Note: It takes 2 transfer cycles to read Bytes 1 and 2
It also takes 2 transfer cycles to read Bytes 3 and 4

Data Alignment with Structures

- **Data padding** (addition of meaningless bytes) is used by compilers to ensure alignment of different data types within a structure.

```
struct rec1 {  
    char c;  
    int i;  
    char d[2];  
}  
rec1 r;
```

No Data Padding	
Address	Memory
0x000	r.c
0x001	
0x002	r.i
0x003	
0x004	
0x005	r.d[0]
0x006	r.d[1]
0x007	
0x008	
0x009	
0x00A	
0x00B	

Data
alignment
violation

Data Padding	
Address	Memory
0x000	r.c
0x001	
0x002	
0x003	
0x004	
0x005	r.i
0x006	
0x007	
0x008	r.d[0]
0x009	r.d[1]
0x00A	
0x00B	

Padded
bytes

Data Alignment with Structures

- **Data padding** (addition of meaningless bytes) is used by compilers to ensure alignment of different data types within a structure.
- Padded bytes are added between end of last structure element and the start of the next to maintain alignment in an array of structures.

```
struct rec1 {  
    char c;  
    int i;  
    char d[2];  
}  
rec1 r;  
:  
rec1 t[10];
```

No Data Padding	
Address	Memory
0x000	r.c
0x001	
0x002	r.i
0x003	
0x004	
0x005	r.d[0]
0x006	r.d[1]
0x007	
0x008	
0x009	
0x00A	
0x00B	

Data alignment violation

Data Padding	
Address	Memory
0x000	r.c
0x001	
0x002	
0x003	
0x004	
0x005	r.i
0x006	
0x007	
0x008	r.d[0]
0x009	r.d[1]
0x00A	
0x00B	

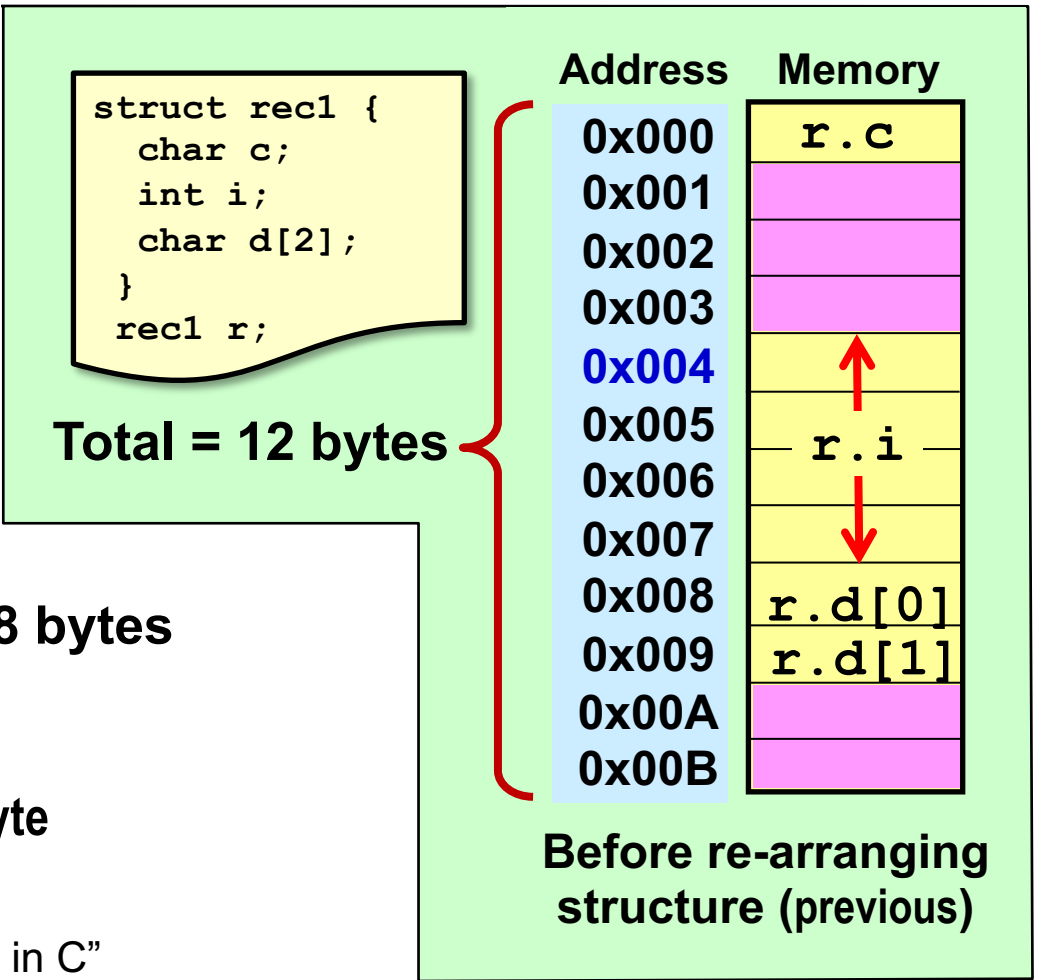
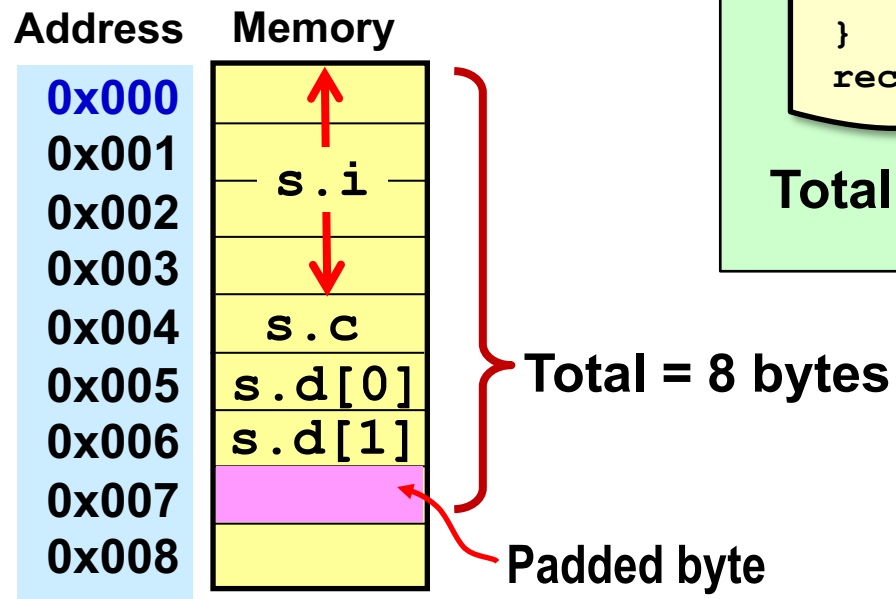
Padded bytes

Data Alignment with Structures (cont)

- Structures should be constructed with data alignment constraints in mind.
- **Rearrange the order** of data objects in the structure based on their size to minimize data padding where possible.

```
struct rec2 {  
    int i;  
    char c;  
    char d[2];  
}  
rec2 s;
```

Data Padding



Summary

- Data alignment restricts where in memory multi-byte data can be stored.
- Data must be aligned to addresses that are multiple of the size of their data type.
- The extent of the data alignment restriction depends on the size of the processor's data bus.
- Structure data types should consider data alignment issues.
- Data padding is used to meet alignment restrictions.

Review Question 1

Assume that we are assigning the address of a pointer to integer in 64-bit data. Which of the following addresses will cause misalignment?

- A) `int *ptr = 0x12345678;`
- B) `int *ptr = 0xDEADBADC;`
- C) `int *ptr = 0x84562516;`
- D) `int *ptr = 0x46543210;`

Answer is C

For the shown struct, what is the correct padding for a 32-bit word machine?

```
struct data {
    char id[5];
    int val;
    char type;
}
rec1 r;
```

Answer is C

(A)

Address	Memory
0x000	r.id[0]
0x001	r.id[1]
0x002	r.id[2]
0x003	r.id[3]
0x004	r.id[4]
0x005	↑
0x006	r.val
0x007	↓
0x008	
0x009	r.type
0x00A	
0x00B	

(B)

Address	Memory
0x000	r.id[0]
0x001	r.id[1]
0x002	r.id[2]
0x003	r.id[3]
0x004	r.id[4]
0x005	
0x006	↑
0x007	r.val
0x008	↓
0x009	
0x00A	r.type
0x00B	

(C)

Address	Memory
0x000	r.id[0]
0x001	r.id[1]
0x002	r.id[2]
0x003	r.id[3]
0x004	r.id[4]
0x005	
0x006	
0x007	
0x008	↑
0x009	r.val
0x00A	↓
0x00B	

r.type

(D)

Address	Memory
0x000	↑
0x001	r.val
0x002	↓
0x003	
0x004	r.id[0]
0x005	r.id[1]
0x006	r.id[2]
0x007	r.id[3]
0x008	r.id[4]
0x009	r.type
0x00A	
0x00B	

Instruction Organisation in Memory

**Characteristics of Instructions
and the
ARM Programmer's Model**

Code and Data in Memory

- Instructions are stored in memory as binary patterns called **machine codes**.
- They are also represented in more readable **mnemonics**.

● ARM (32-bit CPU) example:
Note: ARM instructions are **32 bits** long

MOV R1 ,R0
MOV R2 ,R1
:

- Most instruction formats consist of **Op-code** and **Operands**.

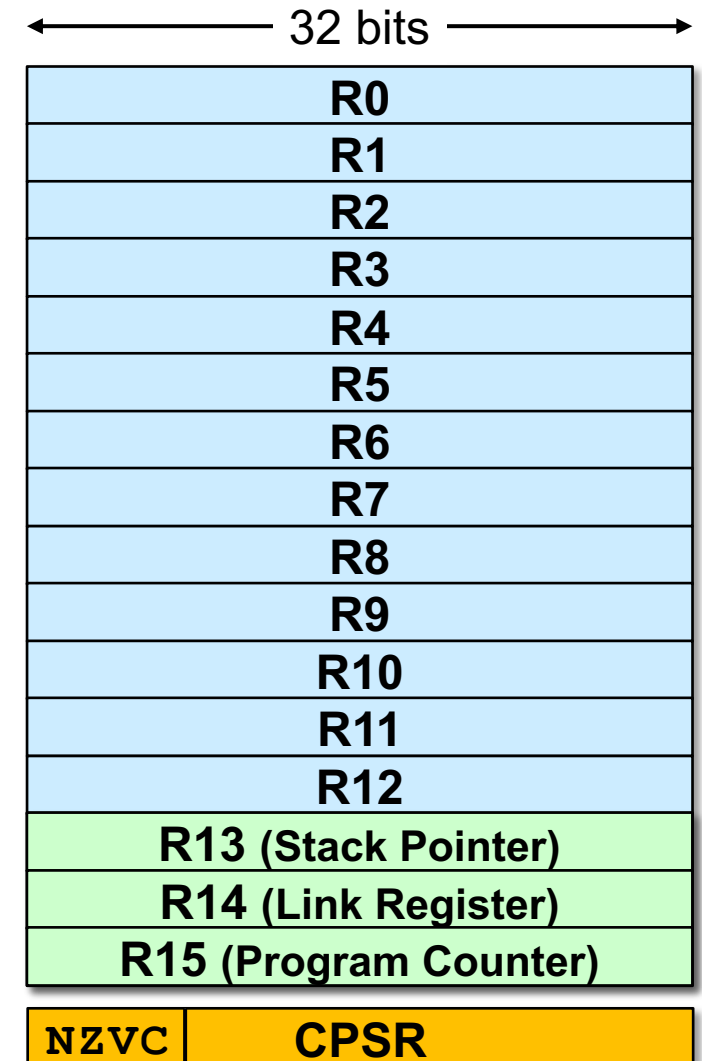


Address	Memory	Mnemonics
0x000	0x00	MOV R1,R0
0x001	0x10	
0x002	0xA0	
0x003	0xE1	
0x004	0x01	MOV R2,R1
:	0x20	
:	0xA0	
:	0xE1	
0x100	0x12	Data
0x101	0x34	Data
:	:	Data Memory
:	:	

Code Memory

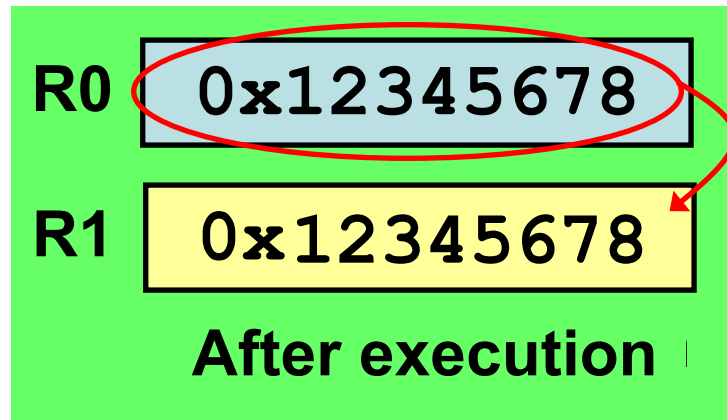
The ARM Programmer's Model

- The ARM processor can operate in various modes. The following are visible registers in the **User mode**.
- There are 16 32-bit registers.
- R0 – R12 are general purpose registers.
- R13 is the Stack Pointer (**SP**).
- R14 is the Link Register (**LR**).
- R15 is the Program Counter (**PC**)
- The Current Processor Status Register (**CPSR**) holds the condition code bits (**NZVC**)



Instructions and Memory accesses

- Instruction execution may involve multiple accesses to memory, resulting in different number of clock cycles to execute the instruction

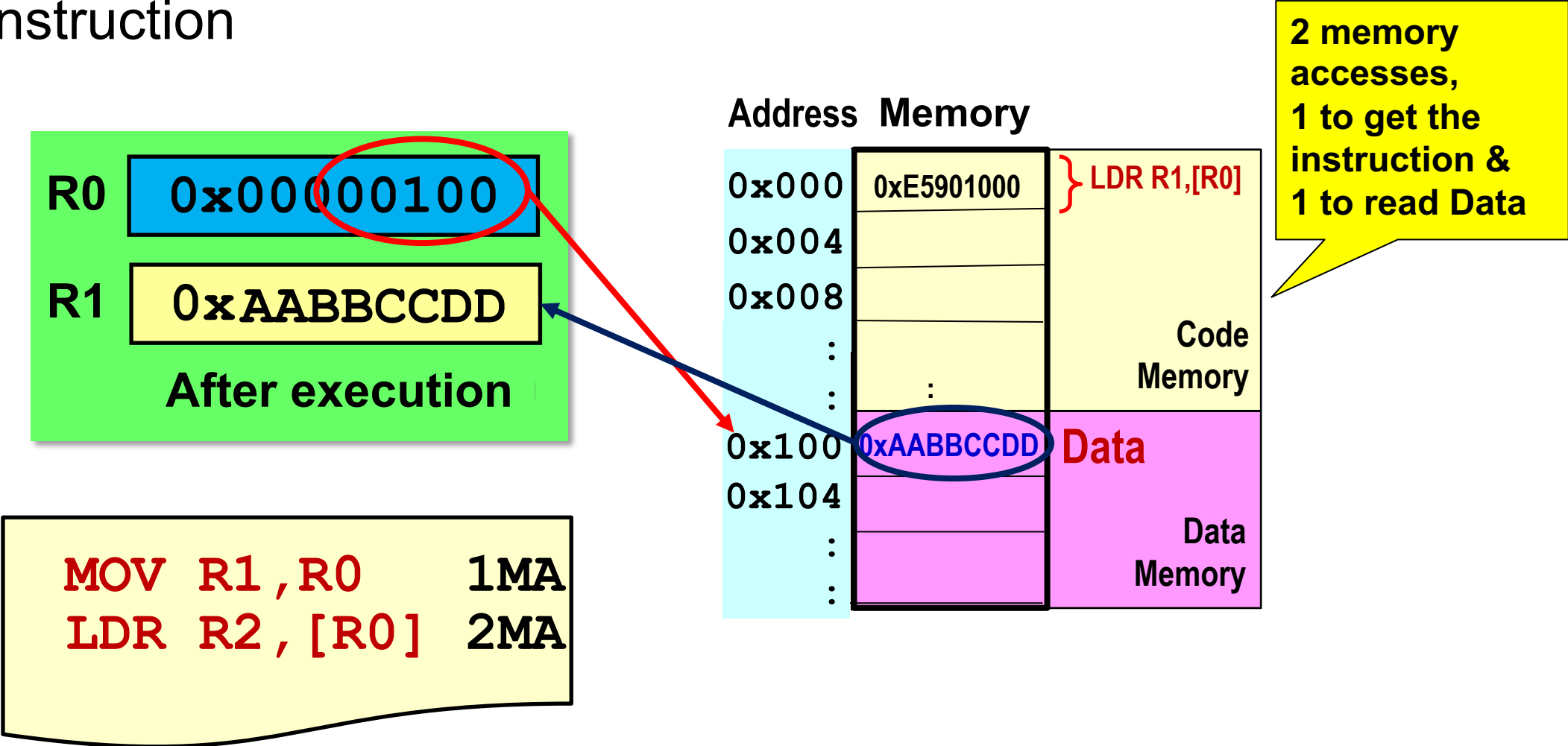


Address	Memory	
0x000	0xE1A01000	} MOV R1,R0
0x004		
0x008		
:	:	
		Code Memory
0x100	0xAABBCCDD	Data
0x104		
:		
:		
		Data Memory

Only 1 memory access, to get the instruction

Instructions and Memory accesses

- Instruction execution may involve multiple accesses to memory, resulting in different number of clock cycles to execute the instruction



Instructions and Memory accesses

- How many memory accesses are consumed in the following code segment

- A) 4
- B) 8
- C) 6
- D) 0

Answer is C

```
MOV R1 ,R0  
LDR R2 , [R0]  
LDR R1 , [R2]  
MOV R0 ,R1
```

Instructions and Memory accesses

- How many memory accesses are consumed in the following code segment, before and after optimization
- A) 3 (optimized 3)
- B) 6 (optimized 4)
- C) 6 (optimized 3)
- D) 3 (optimized 2)

Answer is B

```
LDR R1 , [R0]  
LDR R2 , [R0]  
LDR R3 , [R0]
```

```
LDR R1 , [R0]  
MOV R2 , R1  
MOV R3 , R1
```